
Ensembles

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

Notice

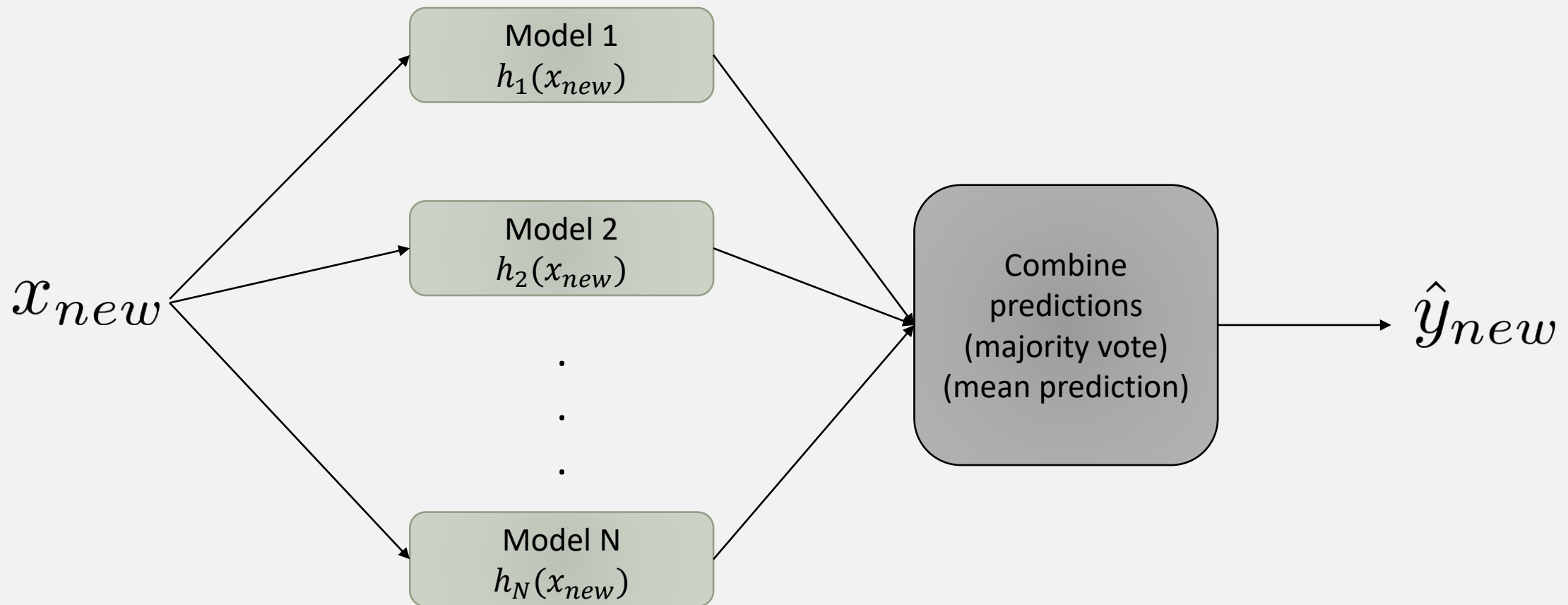
- Slides prepared by Moayad Alnammi – malnammi.github.io
- Slides that adapt figures or content from other sources have a link to the source.

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Outline

- Overview
- Encouraging diversity in errors
- Bootstrap Aggregation (Bagging)
 - Random Forests
- Boosting
 - Adaboost
- Discussion on Bagging and Boosting
- Summary

Trained models take in an instance and output a prediction



An ensemble combines the predictions of multiple trained models to produce a single prediction. We want the ensemble performance to be no worse than any of the single models.

Overview

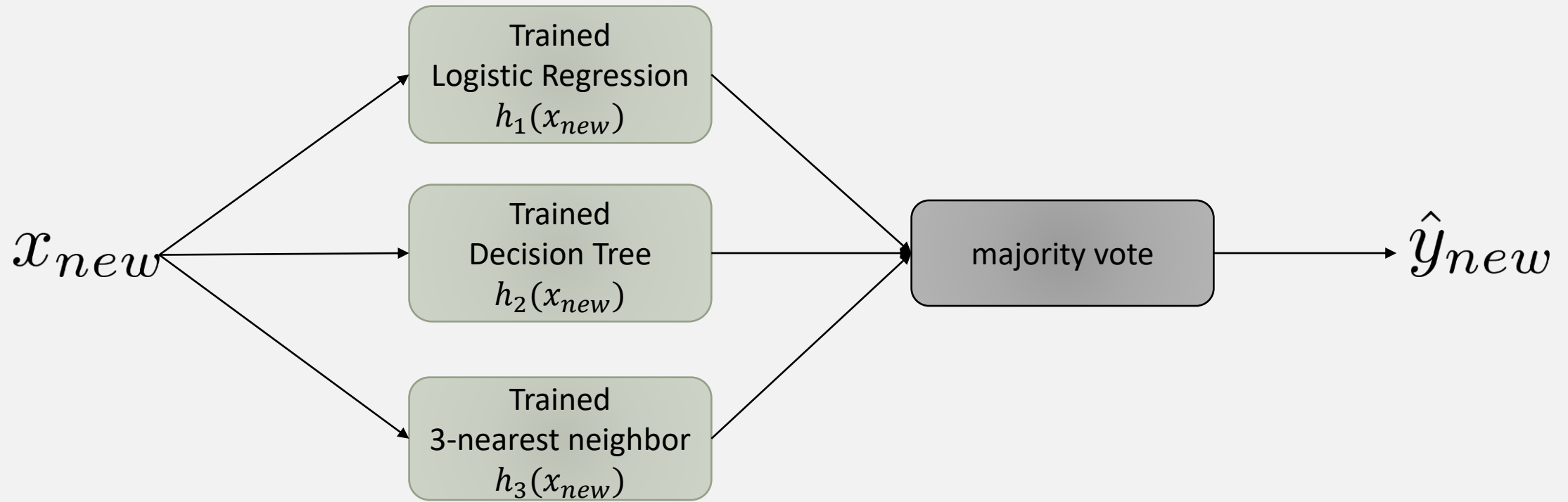
- An ensemble contains N trained learning algorithms (also called **base** learners). When predicting, each of the N models predict, and the N predictions are combined in some way (ensembled).
- We want the ensemble performance to be no worse than any of the single models. Preferably better than each individual model.
- How to encourage ensemble to be better than individual models? When the individual models in the ensemble:
 - make **diverse and uncorrelated (independent)** errors
 - and their performance is better than random guessing
- When combining multiple independent and diverse decisions each of which is better than random guessing, then:
 - correct decisions are encouraged by the majority
 - and random errors (incorrect decisions) are discouraged by the majority

Options to Encourage Diversity in Errors

- It's difficult to get models whose errors are completely uncorrelated, but we can *encourage* diversity in their errors by:
 - **Diversify algorithms:** choose various types of learning algorithms (knn, decision tree, linear regression, neural networks, etc.)
 - **Diversify algorithm settings:** choose various hyperparameter settings for the same algorithm (decision trees with various depth limits and minimum samples at the leaf, linear regression with various regularization settings, etc.).
 - **Bagging:** use the same learning algorithm for each of the N individual models, but train on different *subsamples* of the training data.
 - **Random Forest:** a bagging-based algorithm. Trains N decision trees. Each tree is trained on a subsample of the training data and randomly selects a subset of features for splitting.
 - **Boosting:** use the same learning algorithm for each of the N individual models, but each algorithm is trained sequentially with different sample weights.

Simple Ensemble: Diverse ML Algorithms trained on entire training set

All algorithms were trained on the entire training set.



With a classification task, we can take the majority vote. For a regression task, we can take the mean. The combination procedure can be more complex than this.

Simple ensemble in scikit-learn [1]

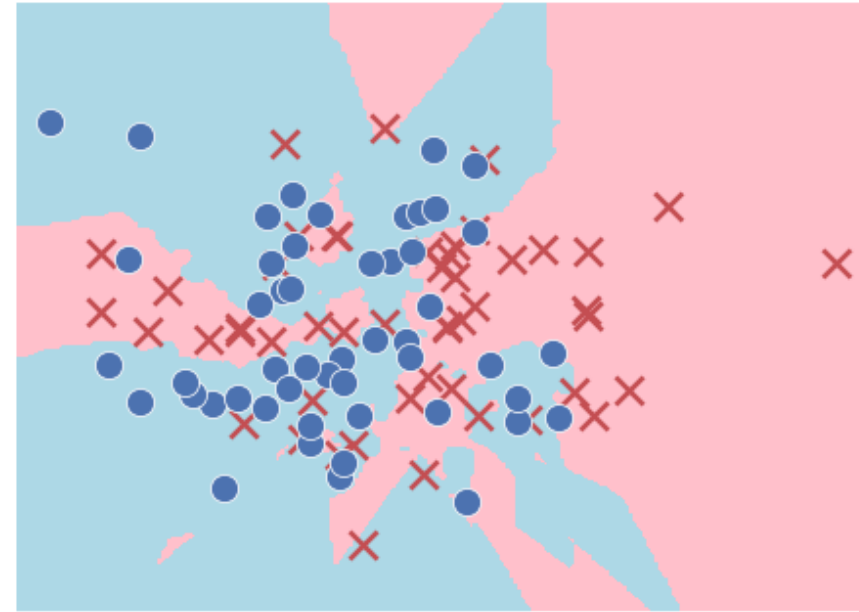
- Scikit-learn's has [VotingClassifier](#) for classification and [VotingRegressor](#) for regression.
- Below is a voting classifier with base learners [logistic regression, 5-nn, decision tree].

```
1  from sklearn.linear_model import LogisticRegression
2  from sklearn.neighbors import KNeighborsClassifier
3  from sklearn.tree import DecisionTreeClassifier
4  from sklearn.ensemble import VotingClassifier
5
6  lr_clf = LogisticRegression()
7  knn_clf = KNeighborsClassifier(n_neighbors=5)
8  dt_clf = DecisionTreeClassifier()
9
10 ensemble_clf = VotingClassifier(estimators=[('lr', lr_clf), ('5-nn', knn_clf), ('dt', dt_clf)], voting='hard')
11 ensemble_clf.fit(X_train, y_train)
```

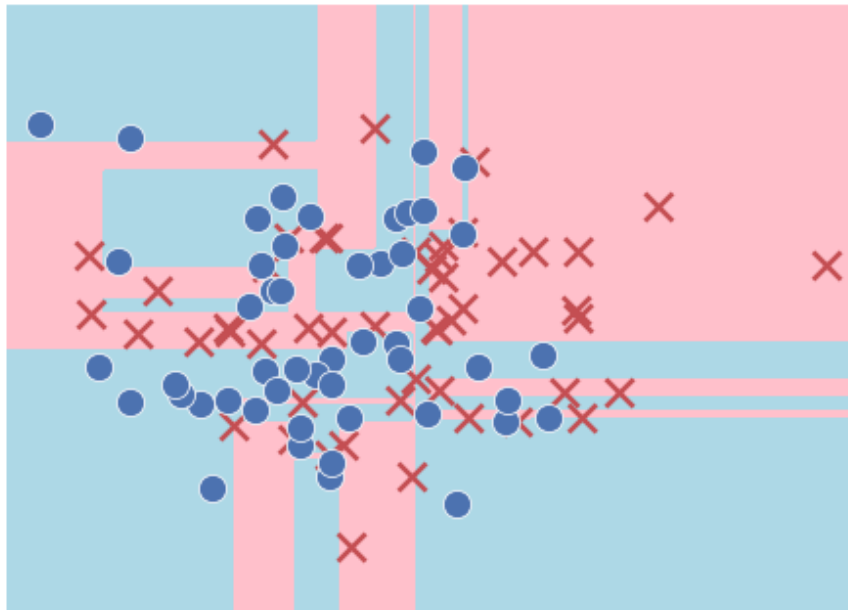
Logistic Regression



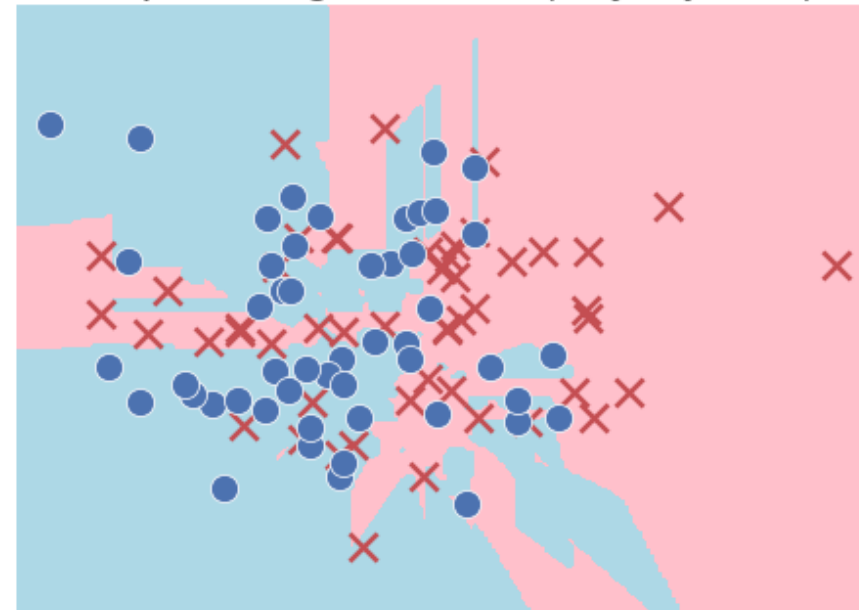
5-nearest neighbor



Decision Tree



Simple Voting Ensemble (Majority Vote)



Bootstrap Aggregation (Bagging) and Random Forests

Bagging Overview

- Bootstrap aggregation (bagging) is an ensemble technique that uses **bootstrap samples** of the training set to train N base learners.
- A bootstrap sample of a dataset S are k instances drawn uniformly at random from S **with replacement**.
 - Can do this with [np.random.choice](#) with argument `replace=True`.
- So, each base learner trains on *different* datasets which promotes diversity in their errors.
- Typically, the N learners are all the same ML algorithm (e.g. logistic regression, decision tree, etc.). But note they will make different predictions since they are trained on different datasets.
- Once trained, we can aggregate the predictions from the N learners via majority vote, mean, or any other form of combination.

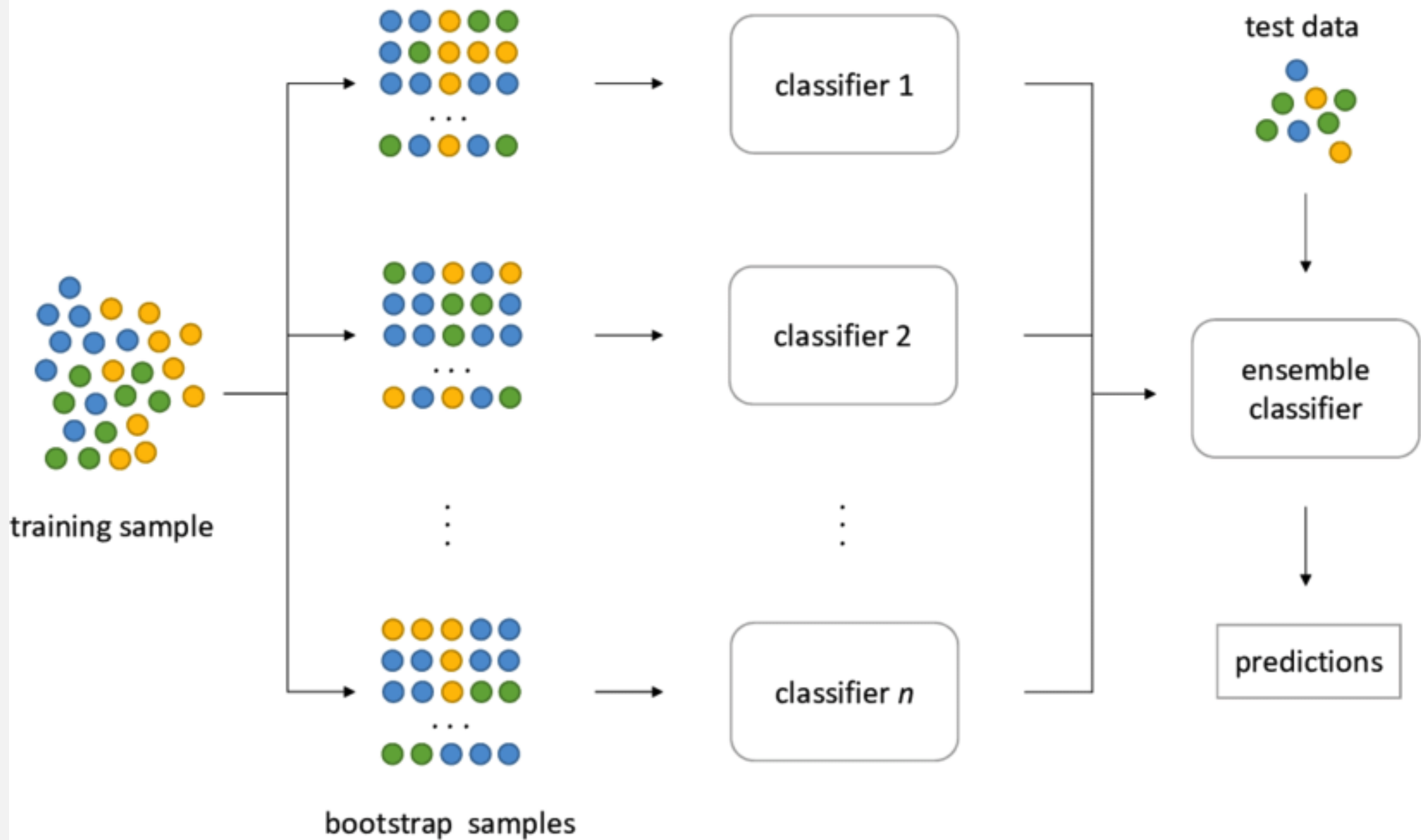


Image
from: [link](#)

- Given a training set $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ and an ML algorithm L , bagging can be done as follows:

Training stage:

for $i = 1, 2, \dots, N$ do:

- Randomly sample k instances from T **with replacement**. This is called a bootstrap sample. Typically $k = m$; the number of training instances.
- Train the algorithm L on the bootstrap sample to produce hypothesis h_i

Prediction stage:

classification: $\hat{y}_{new} = \text{majority_vote}(h_1(x_{new}), h_2(x_{new}), \dots, h_N(x_{new}))$

regression: $\hat{y}_{new} = \frac{1}{N} \sum_{i=1}^N h_i(x_{new})$

Bagging Reduces Variance [1]

- Recall that every ML algorithm's generalization error on a specific problem can be decomposed as:

$$MSE = Bias^2 + Variance + Noise$$

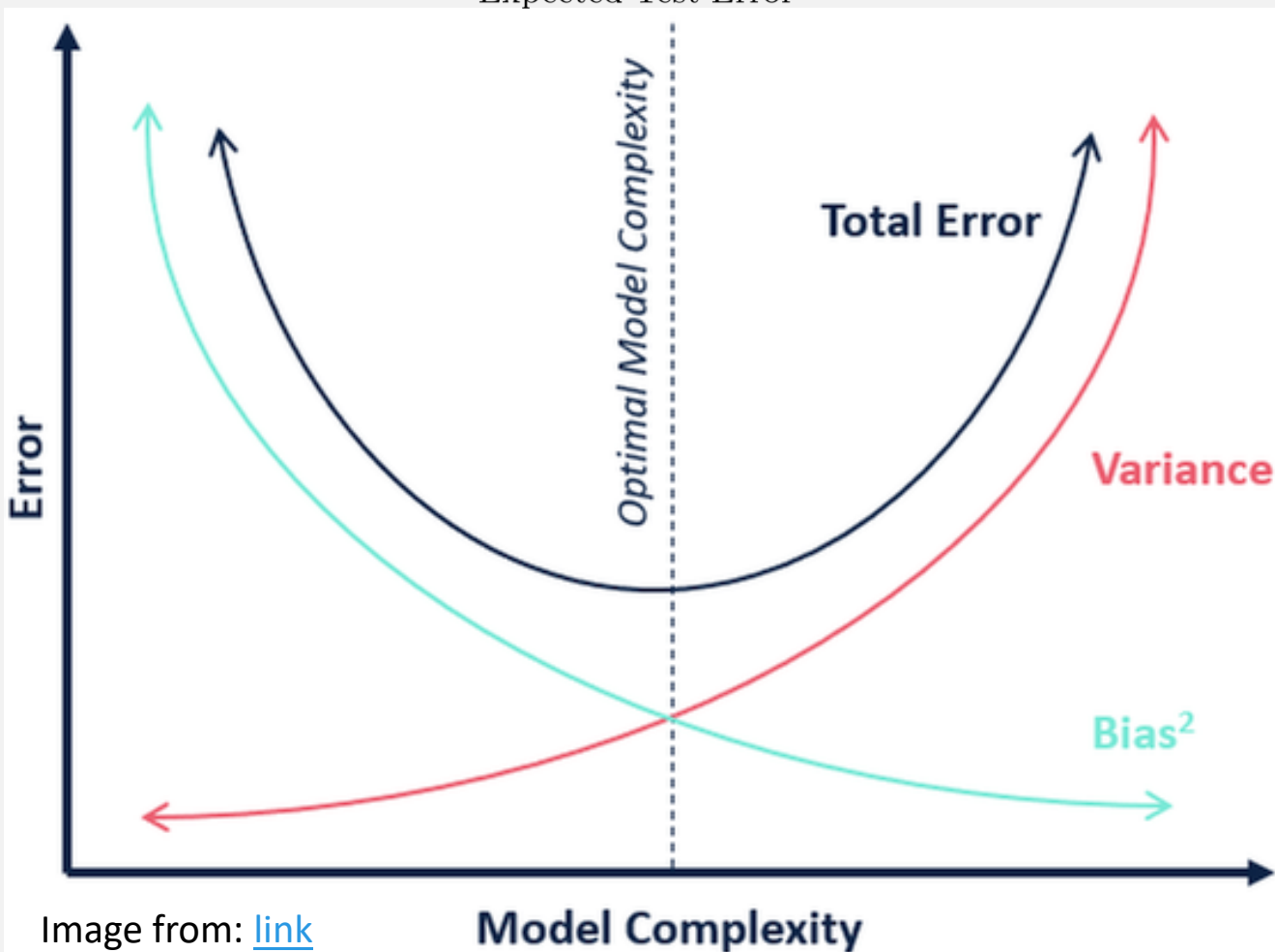
- In bagging we are ensembling N predictions using a base learning algorithm L with a certain MSE for the problem.
- What can we say about the MSE of Bagging?
 - It has the same bias as L .
 - It has lower MSE than L .
 - It reduces the MSE, by **reducing the variance**. The variance becomes $\frac{Variance_L}{N}$
- So, if the algorithm L is complex and *tends* to overfit (low bias, high variance), then bagging can help with combatting overfitting (since it lowers the variance).

Bagging Reduces Variance [2]

- So, if the algorithm L is complex and *tends* to overfit (low bias, high variance), then bagging can help with combatting overfitting (since it lowers the variance).
- The more models we use in the bagging, the more we reduce the variance of L .
- This is the **main takeaway** from bagging: it helps combat overfitting by reducing the variance of the base algorithm.
- Another important computational note about bagging is that the N models can be trained in **parallel**.

Recall the Bias-Variance Tradeoff

$$\underbrace{E_{\mathbf{x},y,D} \left[\left(\frac{1}{N} \sum_{i=1}^N h_i(\mathbf{x}) - y \right)^2 \right]}_{\text{Expected Test Error}} = \underbrace{E_{\mathbf{x}} \left[(\bar{h}(\mathbf{x}) - \bar{y}(\mathbf{x}))^2 \right]}_{\text{Bias}^2} + \underbrace{E_{\mathbf{x},y} \left[(\bar{y}(\mathbf{x}) - y)^2 \right]}_{\text{Noise}} + \underbrace{\text{Var} \left[\frac{1}{N} \sum_{i=1}^N h_i(\mathbf{x}) \right]}_{\text{Variance}}$$



- In general, trying to reduce the bias **tends** to increase the variance, and vice versa.
- In bagging, we reduce the variance while the bias remains unchanged.
- Thus, we can use a complex base learner that has high-variance low-bias (which tends to overfit), and reduce the variance via bagging to get low-variance low-bias (combat overfitting).

$$\begin{aligned} \text{Var} \left[\frac{1}{N} \sum_{i=1}^N h_i(\mathbf{x}) \right] &= \frac{1}{N^2} \text{Var} \left[\sum_{i=1}^N h_i(\mathbf{x}) \right] \\ &= \frac{1}{N} \text{Var} [h(\mathbf{x})] \end{aligned}$$

Random Forests – a Bagging-based method

- This is the **main takeaway** from bagging: it helps combat overfitting by reducing the variance of the base algorithm L .
- Decision trees *tend* to overfit. They are also computationally fast to train.
- Random forests is bagging with decision trees and one more idea:
 - at each node, when deciding which features to split on, it only considers a random subset of features. This further promotes diversity in the errors.
- Training process is the same as in bagging, but when building a decision tree only consider a random subset of the features for splitting. Typically, this random subset is much smaller than the entire set of features.
- Very competitive in practice. Wins a lot of Kaggle competitions on structured datasets (*excel* table type of data).

Bagging and Random Forests in scikit-learn [1]

- Scikit-learn's [bagging](#) implementation allows you to pass a base learner method. Below we use 5 polynomial logistic regression as base learners.

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.ensemble import BaggingClassifier
3 from sklearn.preprocessing import PolynomialFeatures
4
5 poly = PolynomialFeatures(3)
6 clf = BaggingClassifier(base_estimator=LogisticRegression(), n_estimators=5, random_state=0)
7 clf.fit(poly.fit_transform(X_train), y_train)
```

- Scikit-learn's [random forest](#) implementation is fast to run. Below we use 1000 decision trees in the random forest. See the documentation. Can use parallelization with multiple threads via `n_jobs`.

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 clf = RandomForestClassifier(n_estimators=1000, random_state=0)
4 clf.fit(X_train, y_train)
```

Polynomial LogReg #0



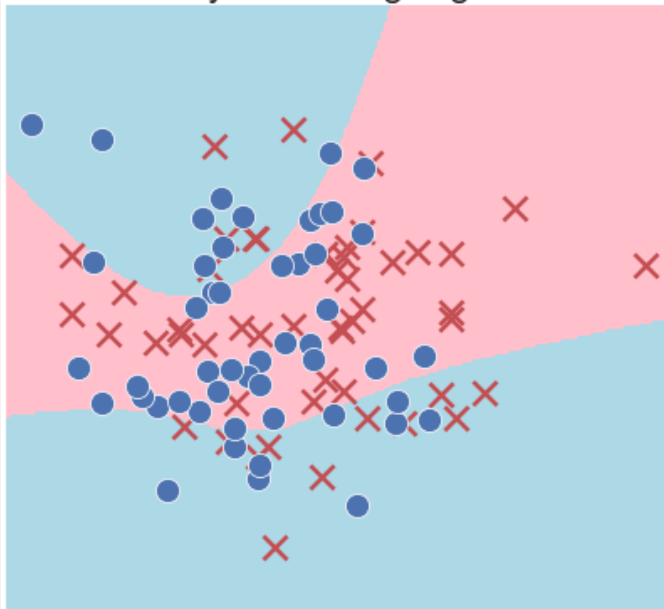
Polynomial LogReg #1



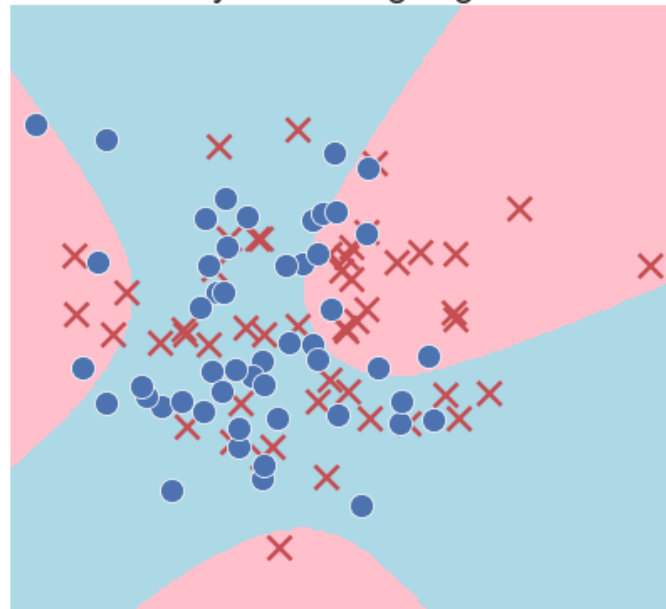
Polynomial LogReg #2



Polynomial LogReg #3



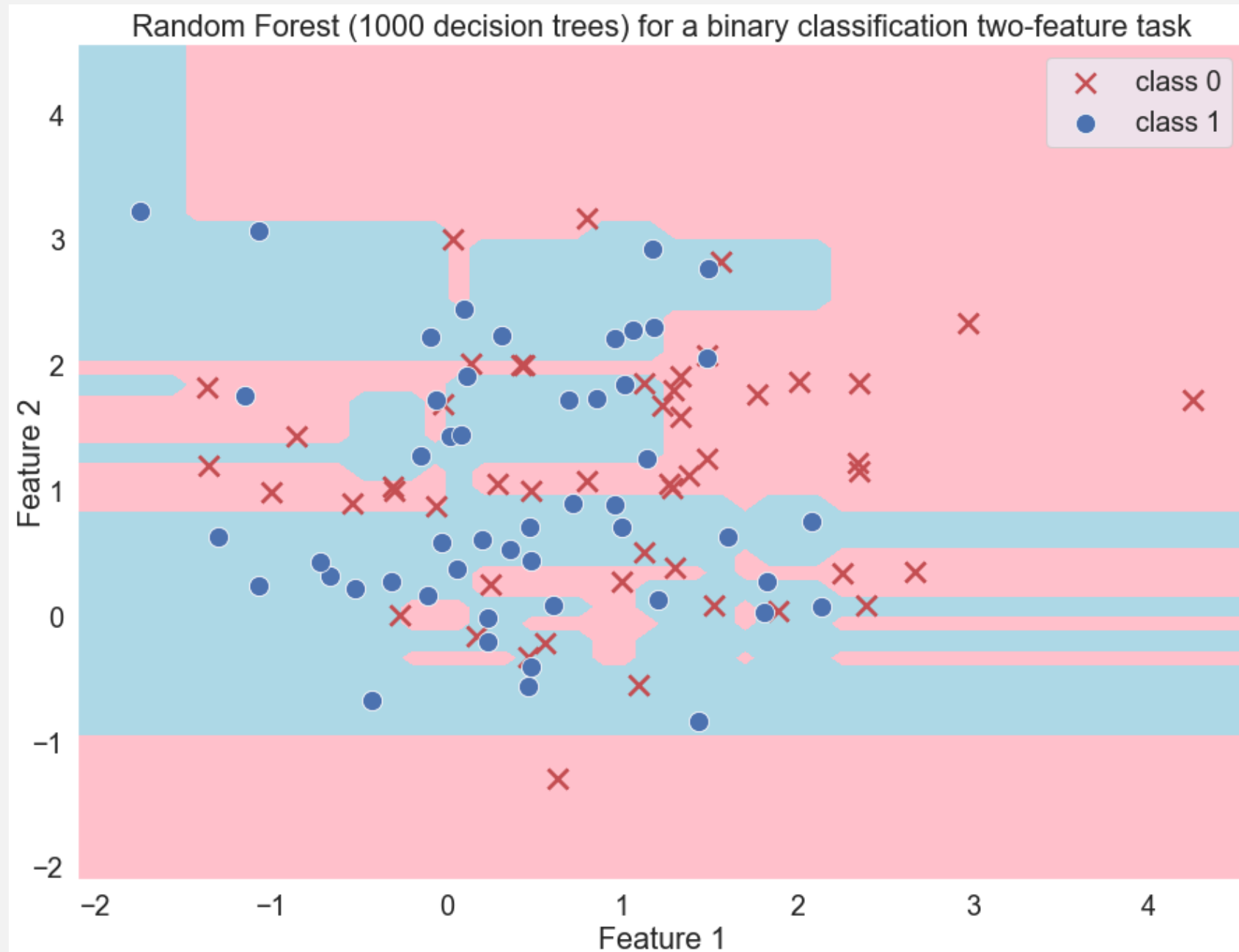
Polynomial LogReg #4



Bagging Ensemble (Majority Vote)



- A random forest with 1000 decision trees was fit to this data.
- Notice the decision surface is much *smoother* than a single decision tree. This is due to the majority vote from 1000 trees.



Boosting and Adaboost

Boosting Overview

- Boosting is an ensembling technique that can help turn a *weak (simple)* learning algorithm into a *strong (complex)* learner.
- The simple algorithm has high-bias low-variance, and boosting helps in reducing the bias.
- In other words, it can help an underfitting learning algorithm perform better on a task.
- In boosting, we have N base learners of algorithm L . But each base learner is trained **sequentially** with emphasis on correcting the mistakes made by the previously trained learners.
- That is, base learner i is trained with more emphasis on the incorrect predictions made by previously trained learners $j < i$.
- This contrasts with what we did in bagging. In bagging we trained N base learners independently. So, in bagging we can train them in parallel.
- We'll cover the idea behind one of the earliest popular boosting algorithms AdaBoost.

Adaptive Boosting (AdaBoost) [Freund & Schapire, 1997]

- Adaptive Boosting (AdaBoost) was first practical boosting method to transform a weak learner L into a strong learner. It is used for classification.
- A weak learner is one that does *slightly* better than random guessing. For example, a tree with just 1-split at the root (called a decision stump).
- Given a training set and a weak learner L , train N learners on the training set sequentially (one after the other). In each iteration you reweigh the training set to give more weight to instances that were misclassified (incorrect).
- In other words, with each iteration you are correcting the mistakes.
- This can be done by having the learner L minimize a *weighted* performance measure. For example, minimizing the weighted logloss:

$$\text{weighted logloss} = \sum_{i=1}^m w_i [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

 weight given to instance i

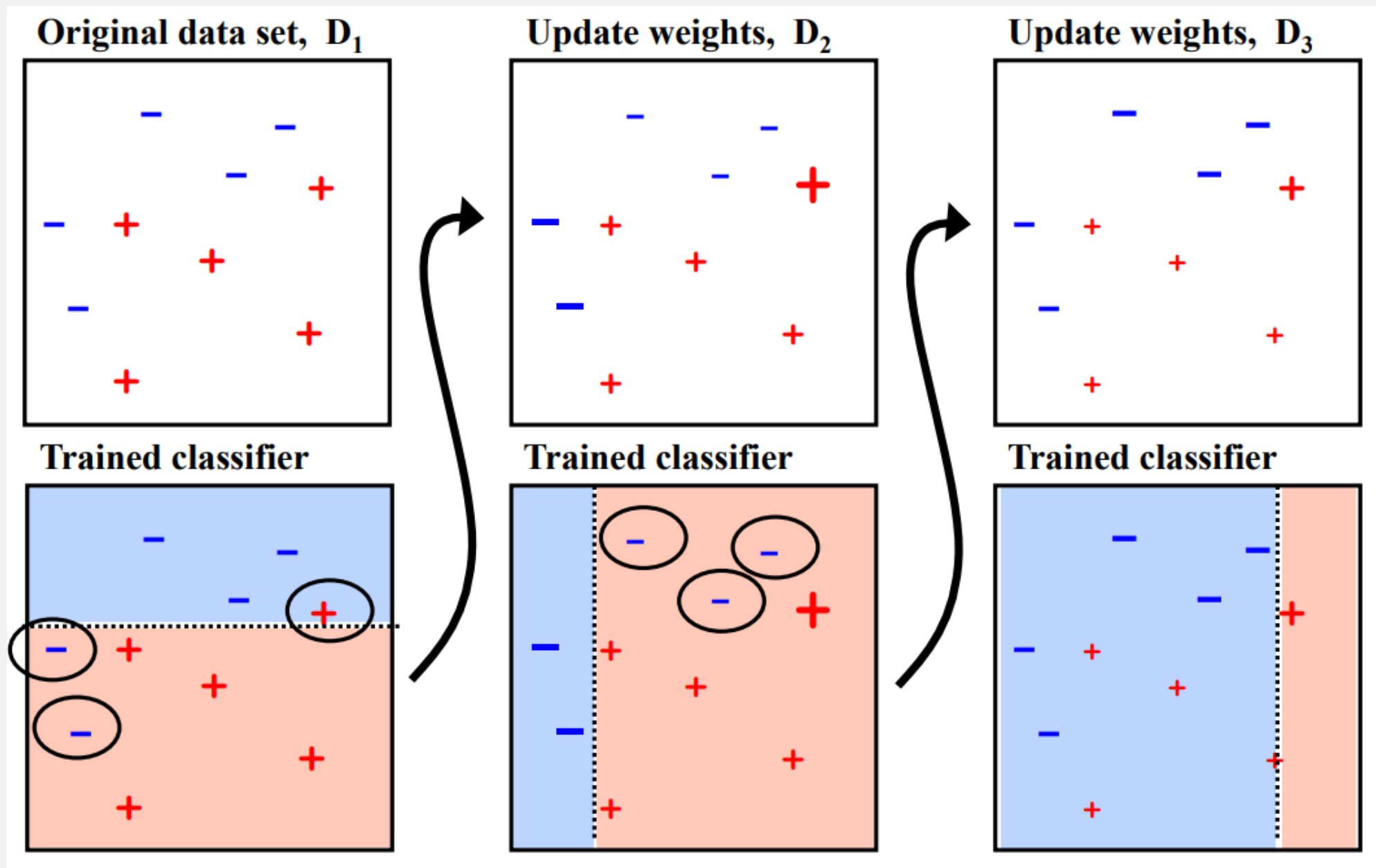


Image from: [link](#)

Given a training set $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ where $y_i \in \{-1, 1\}$, and an ML algorithm L that outputs $\{-1, 1\}$.

Training stage:

initialize the weights of the training instances: $w_i = 1/m$. weights sum to 1.

for $t = 1, 2, \dots, N$ do:

1. Produce hypothesis h_t by training L on train-set with weights w_i .
2. After training, compute the weighted error:
$$\epsilon_t = \sum_{i=1}^m w_i \begin{cases} 1 & \text{if } h_t(x_i) \neq y_i \\ 0 & \text{if } h_t(x_i) = y_i \end{cases}$$
3. Assign hypothesis coefficient α_t such that a hypothesis that has smaller ϵ_t gets larger α_t . This gives more prediction contribution to better hypotheses.
4. Update weights w_i using α_t such that incorrect predictions increase in weight, and correct predictions decrease in weight.
5. re-normalize the weights so that they sum to 1. Set $C := \sum_{i=1}^N w_i$, then $w_i := \frac{w_i}{C}$.

AdaBoost Algorithm – Prediction

- Recall AdaBoost is considering $y_i \in \{-1, 1\}$, and an ML algorithm L that outputs $\{-1, 1\}$.
- Once the training stage is done, you can perform a prediction by taking weighted predictions of the N learners:

$$\hat{y}_{new} = \text{sign} \left[\sum_{t=1}^N \alpha_t h_t(x_{new}) \right] \quad \text{where} \quad \text{sign}(a) = \begin{cases} 1 & \text{if } a > 0 \\ 0 & \text{if } a = 0 \\ -1 & \text{if } a < 0 \end{cases}$$

- Recall that α_t gives more weight to hypotheses that have lower error ϵ_t .

AdaBoost Illustration

- Illustration of AdaBoost on a training set with two classes. Here we train 3 decision stumps sequentially.
- Iter #1: all training instances have equal weight. Learnt hypothesis gets two incorrect blue instances.
- Iter #2: the two incorrect blue instances from iter #1 are upweighted. Learnt hypothesis gets three incorrect.
- Iter #3: the three incorrect instances from iter #2 are upweighted, with downweighing the 2 blue that it got correct.
- [4] shows the resulting AdaBoost ensemble.

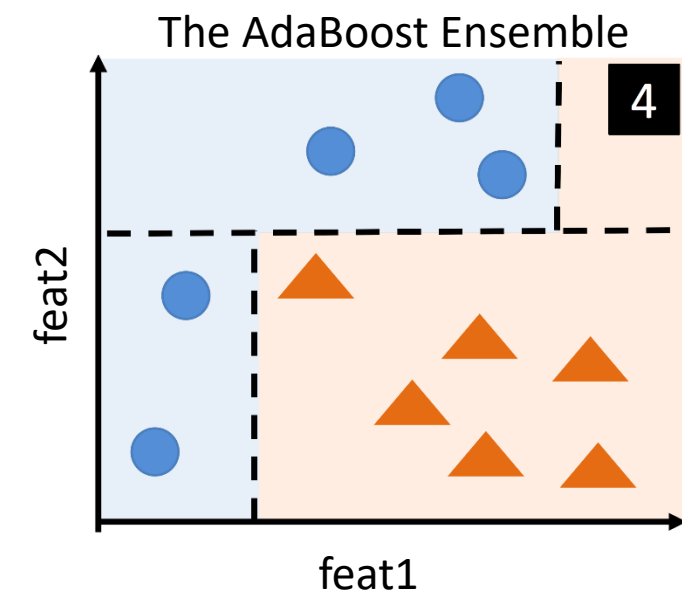
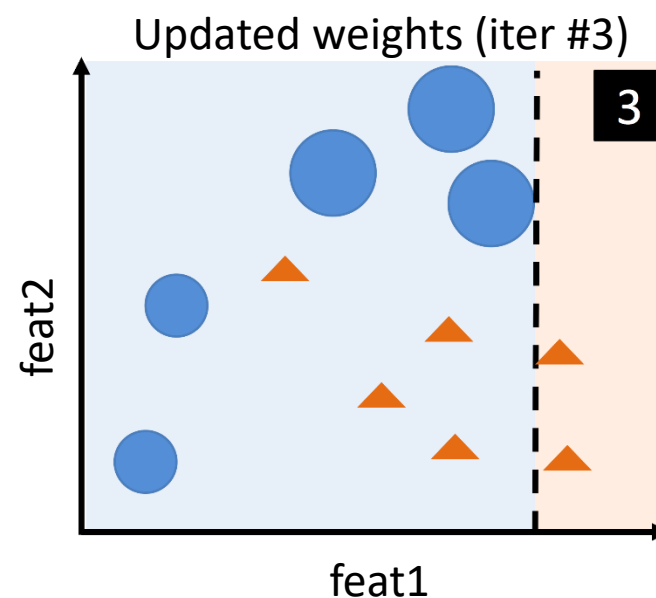
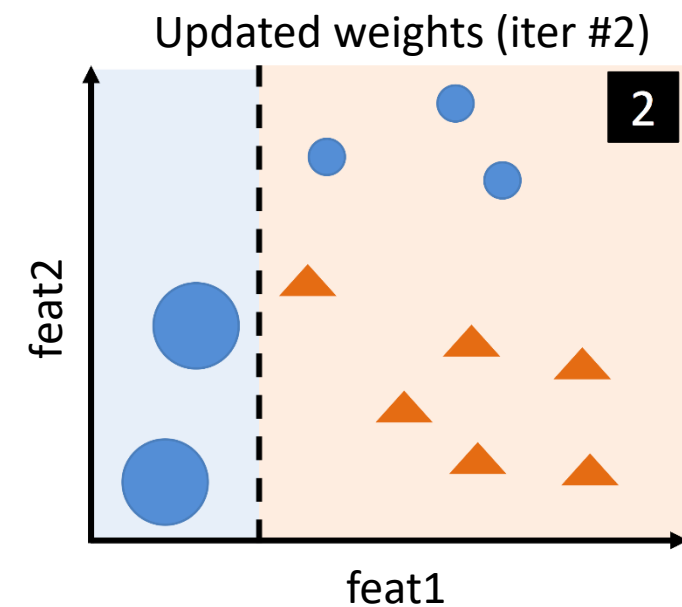
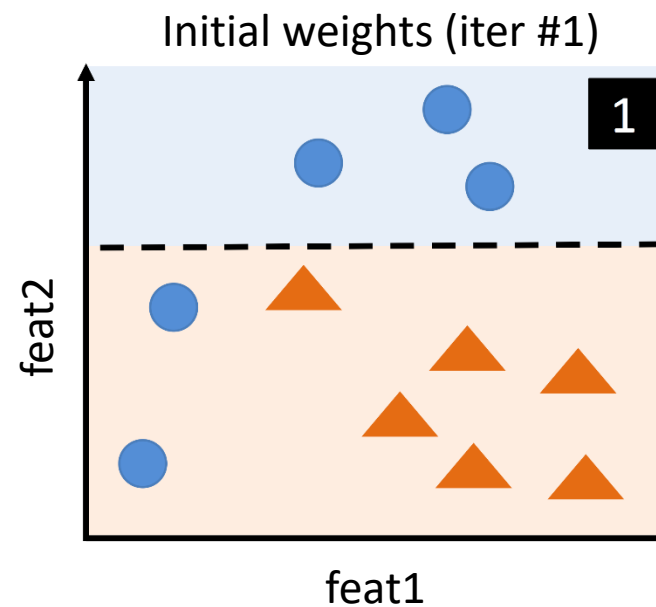


Image from: [link](#)

Given a training set $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ where $y_i \in \{-1, 1\}$, and an ML algorithm L that outputs $\{-1, 1\}$.

Training stage:

initialize the weights of the training instances: $w_i = 1/m$. weights sum to 1.

for $t = 1, 2, \dots, N$ do:

1. Produce hypothesis h_t by training L on train-set with weights w_i .
2. After training, compute the weighted error:
$$\epsilon_t = \sum_{i=1}^m w_i \begin{cases} 1 & \text{if } h_t(x_i) \neq y_i \\ 0 & \text{if } h_t(x_i) = y_i \end{cases}$$
3. Assign hypothesis coefficient $\alpha_t = \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}$. A hypothesis that gets lower error gets larger coefficient; used also in prediction.
4. Update weights: $w_i := w_i e^{-y_i \alpha_t h_t(x_i)}$. Incorrect predictions increase in weight, correct predictions decrease in weight.
5. re-normalize the weights so that they sum to 1. Set $C := \sum_{i=1}^N w_i$, then $w_i := \frac{w_i}{C}$.

AdaBoost Algorithm – Prediction

- Once the training stage is done, you can perform a prediction by taking weighted predictions of the N learners:

$$\hat{y}_{new} = \text{sign} \left[\sum_{t=1}^N \alpha_t h_t(x_{new}) \right] \quad \text{where} \quad \text{sign}(a) = \begin{cases} 1 & \text{if } a > 0 \\ 0 & \text{if } a = 0 \\ -1 & \text{if } a < 0 \end{cases}$$

- Recall that α_t gives more weight to hypotheses that have lower error ϵ_t .
- Another note, we can work with $\{0, 1\}$ labels by equivalently using the weight updates:

$$w_i := \begin{cases} w_i e^{-\alpha_t} & \text{if } x_i \text{ correctly predicted} \\ w_i e^{\alpha_t} & \text{if } x_i \text{ incorrectly predicted} \end{cases}$$

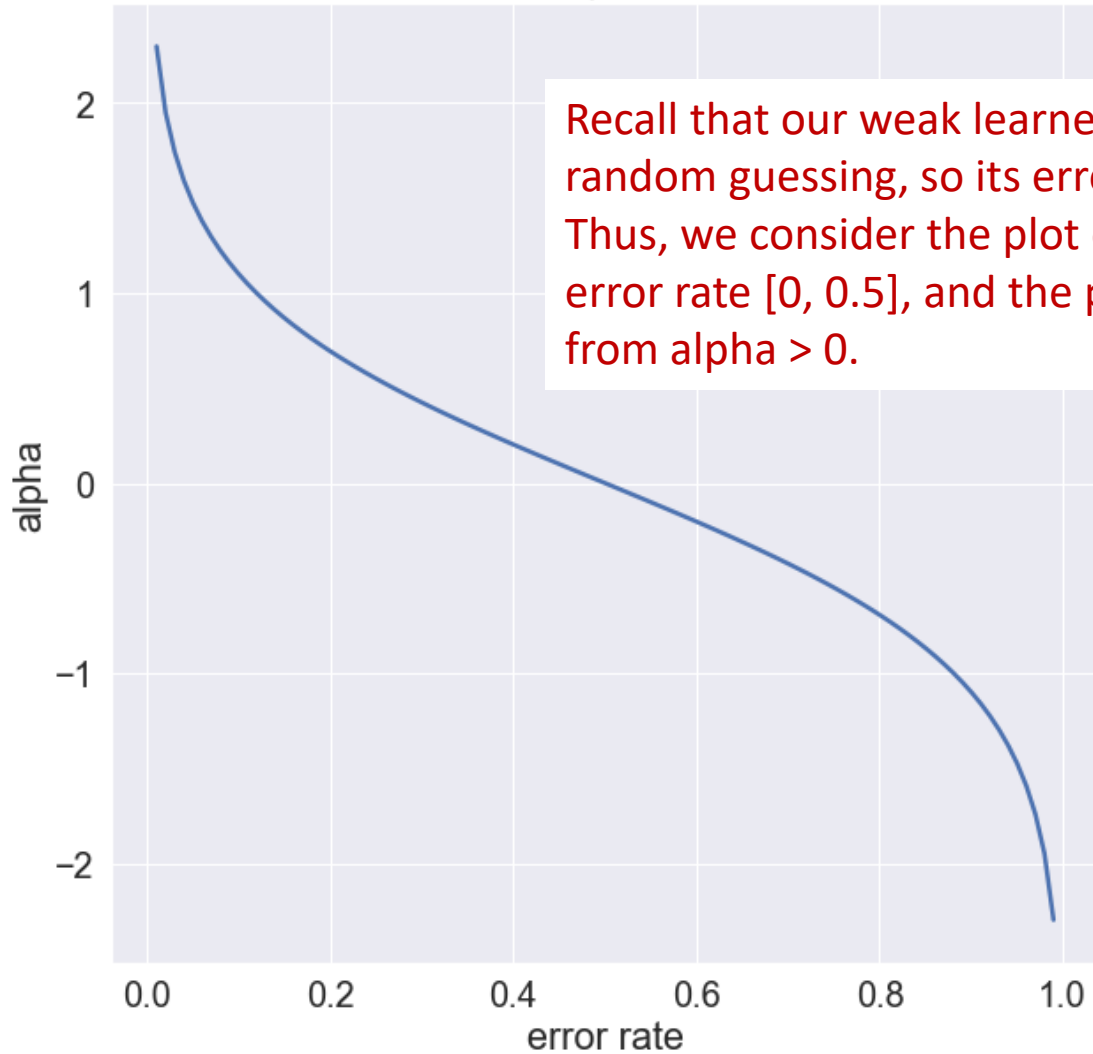
$$\alpha_t = \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}$$

$$\hat{y}_{new} = \text{sign} \left[\sum_{t=1}^N \alpha_t h_t(x_{new}) \right]$$

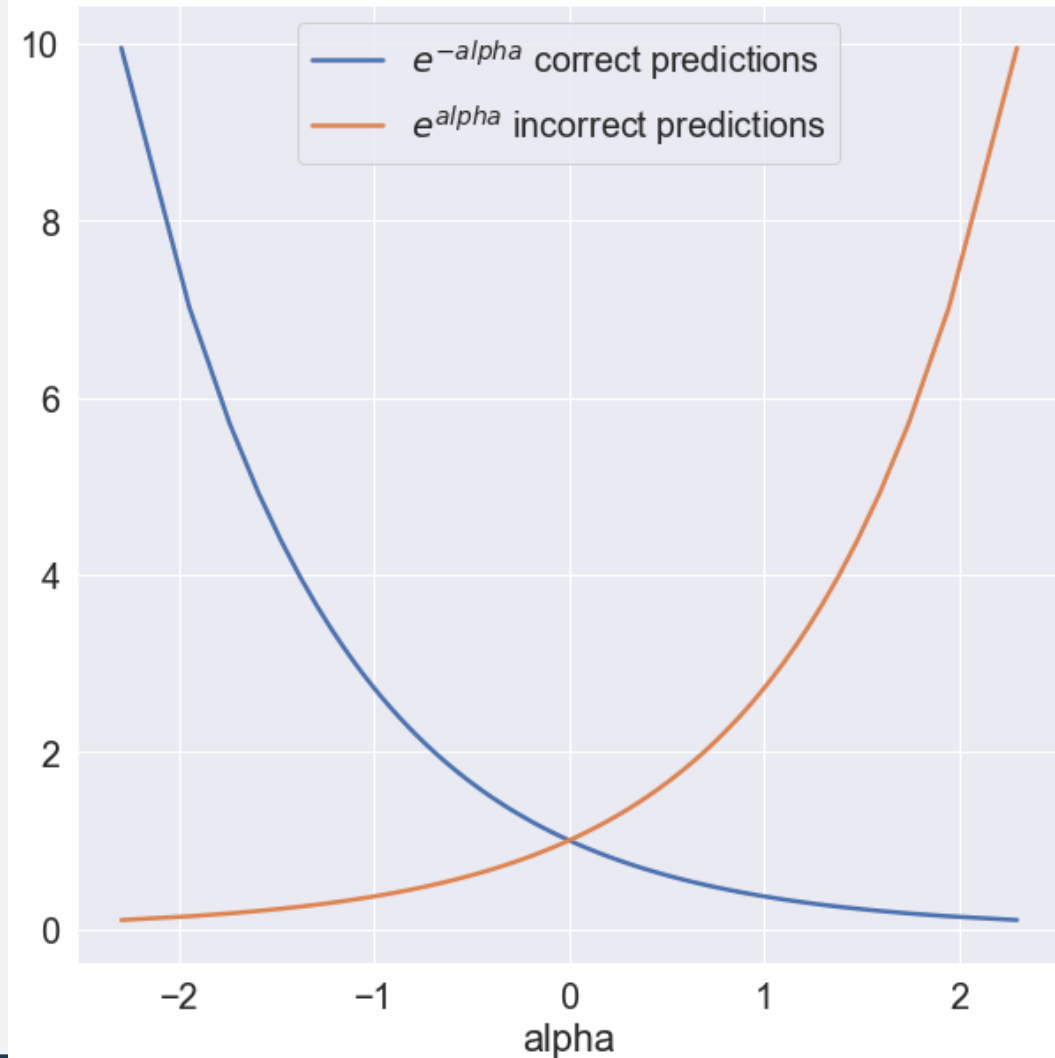
$$w_i := \begin{cases} w_i e^{-\alpha_t} & \text{if } x_i \text{ correctly predicted} \\ w_i e^{\alpha_t} & \text{if } x_i \text{ incorrectly predicted} \end{cases}$$

where $\epsilon_t \in [0, 1]$, so $\alpha_t \in (-\infty, \infty)$

error rate vs alpha in AdaBoost



alpha vs $e^{(-)\alpha}$ in AdaBoost

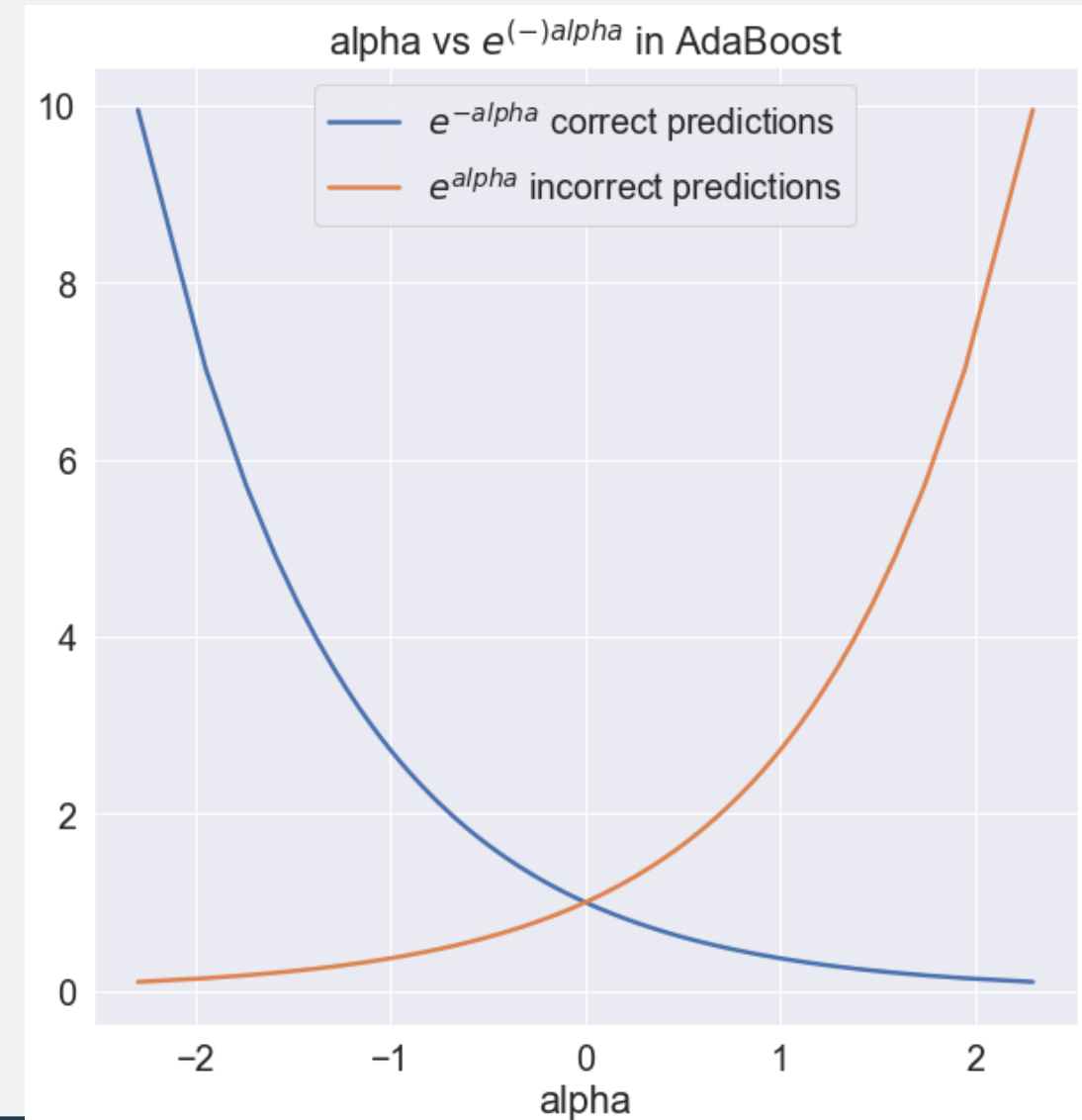


$$\hat{y}_{new} = \text{sign} \left[\sum_{t=1}^N \alpha_t h_t(x_{new}) \right]$$

$$\alpha_t = \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}$$

$$w_i := \begin{cases} w_i e^{-\alpha_t} & \text{if } x_i \text{ correctly predicted} \\ w_i e^{\alpha_t} & \text{if } x_i \text{ incorrectly predicted} \end{cases}$$

- From the plots and formulas, there are two cases:
- Case 1: with weak learners (error rate < 0.5), the smaller the error rate, the higher the alpha. So a good model will contribute more in the ensemble since alpha will be high. Correspondingly, the weights will **increase** on **incorrect** predictions and **decrease** on **correct** predictions.
- Case 2: if a model gets error rate > 0.5, then alpha will be a negative value. This means that it will contribute negatively to the ensemble. In other words, since the model is doing bad, we will take the opposite of its prediction (i.e. if h_t has error rate < 0.5, then taking $-h_t$ will get us error rate > 0.5). Correspondingly, since we take the opposite prediction of the bad model ($-h_t$), the weights will **increase** on **correct** predictions and **decrease** on **incorrect** predictions.



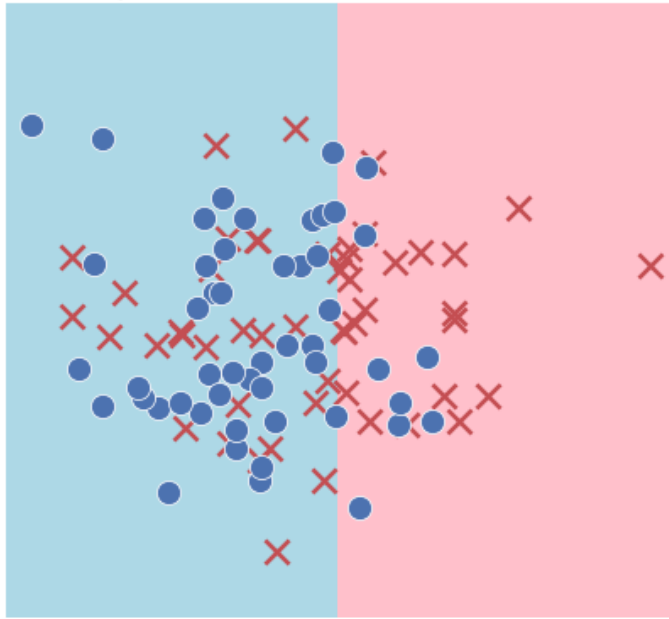
AdaBoost in scikit-learn [1]

- Scikit-learn's [adaboost](#) implementation allows you to pass a base learner method (needs support for sample-weights). The default base learner used is a decision tree with depth 1 (decision stump).
- Below I train a 5 ensemble AdaBoost with 1-split decision tree as base learner.

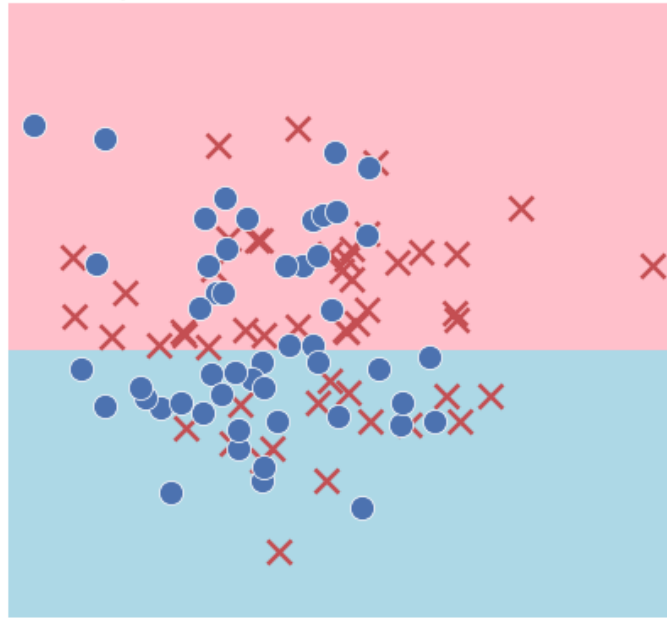
```
1 from sklearn.ensemble import AdaBoostClassifier
2
3 clf = AdaBoostClassifier(n_estimators=5, random_state=0)
4 clf.fit(X_train, y_train)
```

- After fitting, you can:
 - Get the individual models using `clf.estimators_`
 - Get α_t using `clf.estimator_weights_`
 - Get error rates ϵ_t using `clf.estimator_errors_`

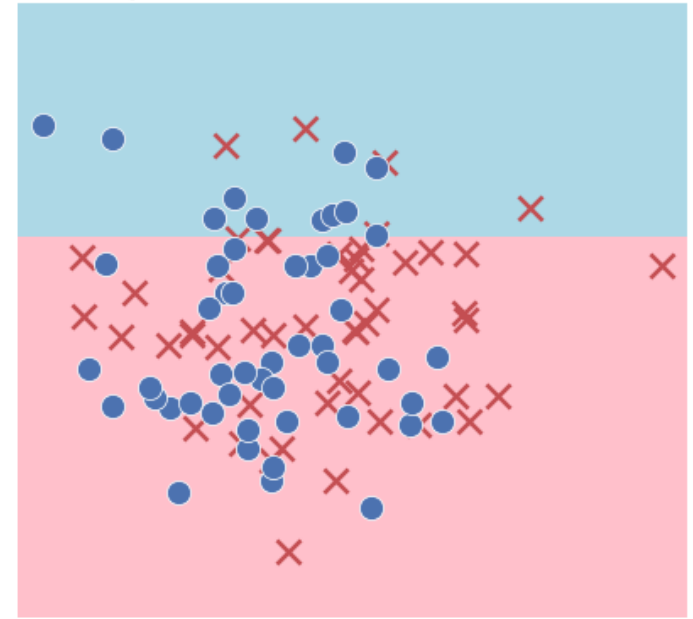
1-Split Decision Tree #0. Acc: 0.65



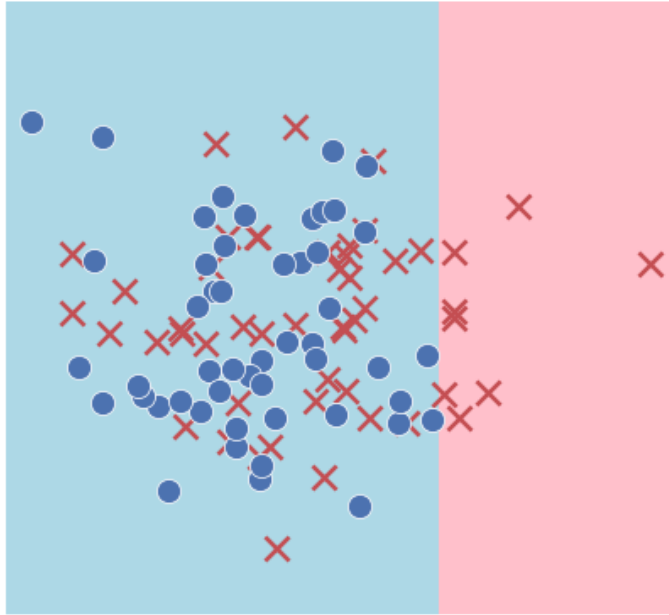
1-Split Decision Tree #1. Acc: 0.62



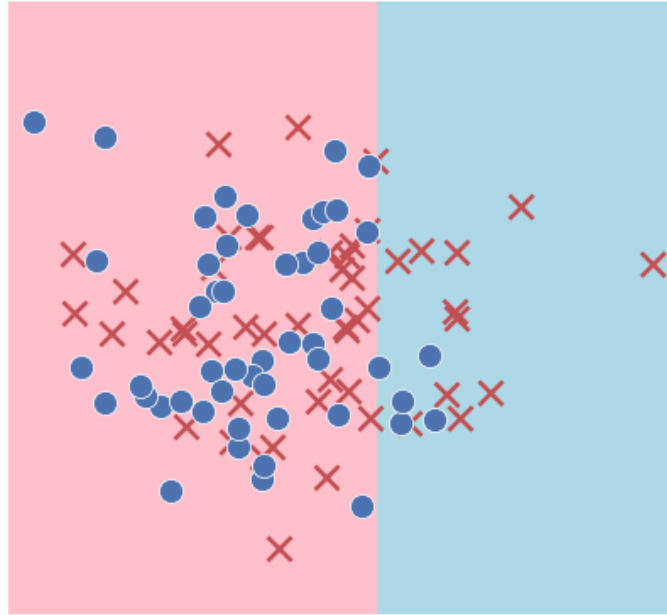
1-Split Decision Tree #2. Acc: 0.56



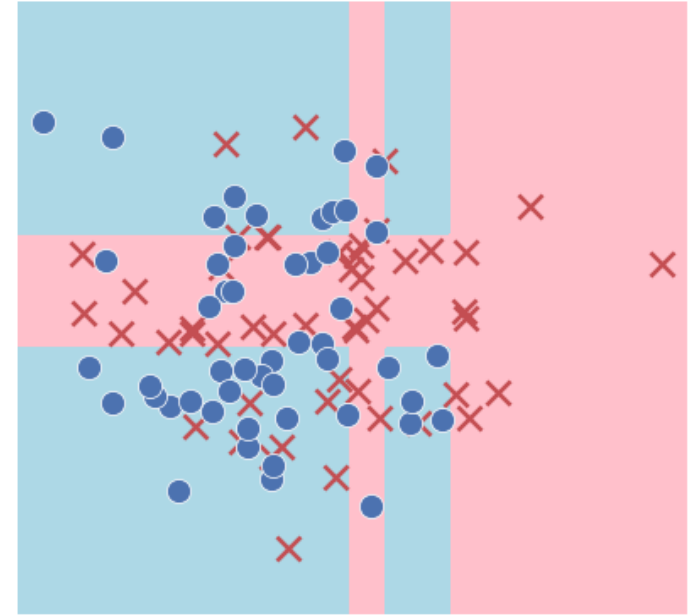
1-Split Decision Tree #3. Acc: 0.58



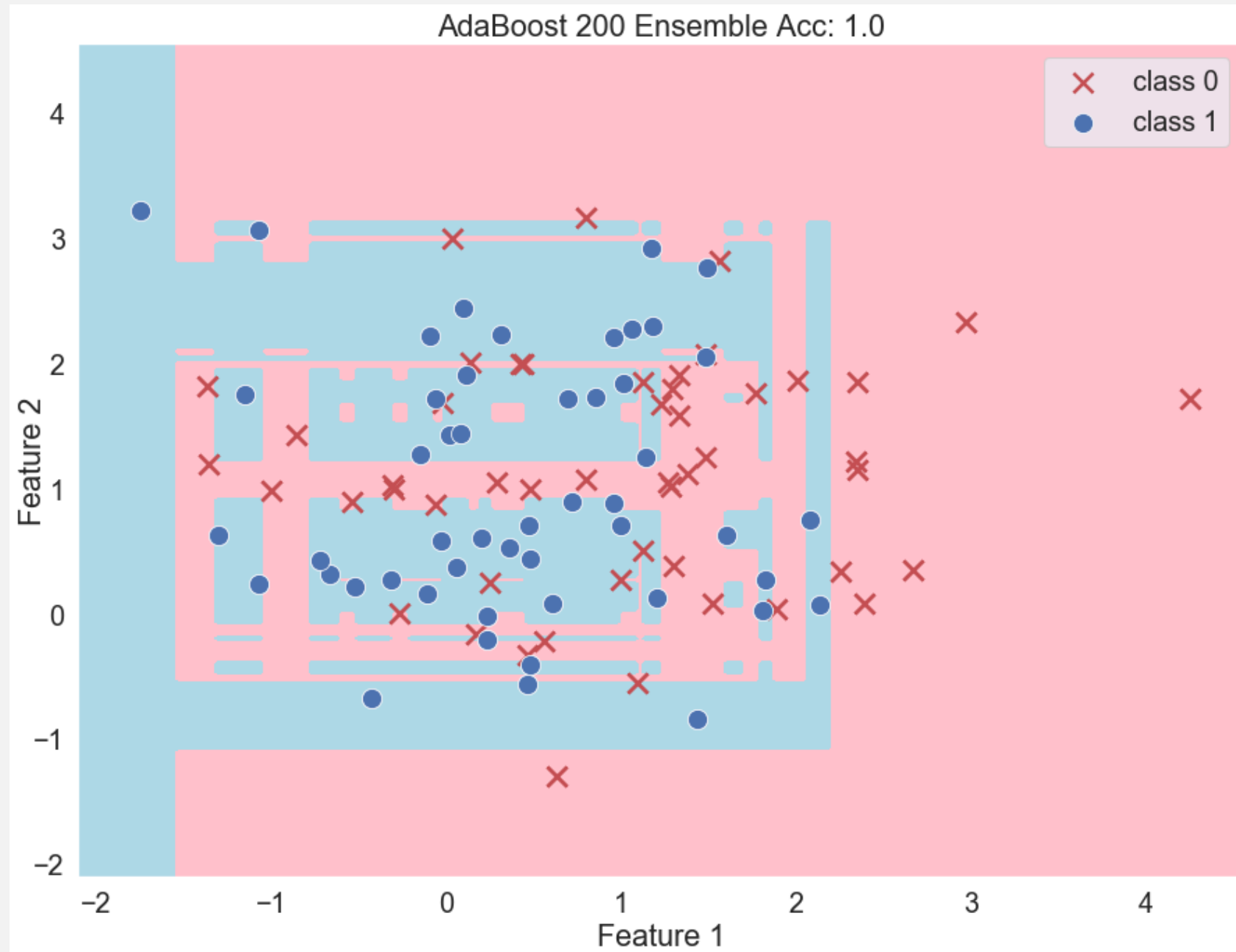
1-Split Decision Tree #4. Acc: 0.44



AdaBoost 5 Ensemble Acc: 0.73



- If I train with 200 base learners, adaboost can get 100% accuracy on the training set.
- It turned a weak learner (1-split decision tree) into a strong learner.
- You have to take care with Boosting, as more iterations (learners) can result in overfitting. However, modern boosting methods like [XGBoost](#) use regularization by default.



Remarks on Bagging and Boosting

- In this class we've discussed two main ensembling techniques: bagging and boosting.
- Studies on various supervised learning tasks have shown that bagging and boosting do better than an individual base learner.
- What about bagging versus boosting? Boosting did better than bagging on most problems, but for some problems bagging did better.
- Bagging can be used to combat overfitting of a strong (complex) algorithm. The complex algorithm has low-bias high-variance, and bagging helps in reducing the variance.
- Whereas boosting helps an underfitting weak (simple) learner. The simple learner has high-bias low-variance, and boosting helps in reducing the bias.
- Both require training N base learners. However, bagging can be run in parallel, whereas boosting is run sequentially.
- There are other ensembling methods like gradient boosting, stacked generalization, etc. Gradient boosting, which is popular nowadays, optimizes on the gradients of a loss function. [XGBoost](#) is a popular gradient boosting algorithm.
- Most Kaggle competitions won by random forest and XGBoost.